



T.C.
MARMARA
UNIVERSITY



FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

CSE4088 Introduction to Machine Learning
Term Project - Final Project Report

Project Title

Email Spam Detection

Group#7 Members

Emir Büçkün - 150119024

Mehmet Sina Çağlar - 150123821

Fatma Melisa Küçük - 150119916

İrem Kubalas - 150119721

Ayşenur Tüfekçi - 150119699

Instructor

Assoc. Prof. Murat Can Ganiz

Table of Contents

1. Abstract.....	3
2. Introduction.....	3
3. Methodology.....	3
3.1. Data Collection and Preprocessing.....	3
3.1.1. Data Cleaning.....	4
3.1.2. Exploratory Data Analysis (EDA).....	4
3.1.3. Feature Engineering and Selection.....	5
3.2. Model Development.....	6
3.2.1. Algorithm Selection.....	6
3.2.2. Model Training and Testing.....	6
3.3. Model Evaluation and Optimization.....	6
3.3.1. Performance Metrics.....	6
3.3.2. Model Optimization.....	7
4. Results.....	7
4.1. Data Preprocessing.....	7
4.1.1. First Few Rows of the Cleaned Dataset.....	7
4.1.2. Basic Information About the Cleaned Dataset.....	7
4.1.3. Message Length Statistics.....	8
4.1.4. Class Distribution.....	9
4.1.5. Relationship Between Message Length and Label.....	10
4.1.6. Top 10 Most Frequent Words in Cleaned Messages.....	10
4.2. Feature Engineering.....	11
4.2.1. Message Length.....	11
4.2.2. Word Count.....	11
4.2.3. Punctuation Count.....	12
4.2.4. Capital Letter Ratio.....	12
4.2.5. Spam Keyword Count.....	12
4.3. Model Performance Evaluation.....	13
4.3.1. Accuracy.....	13
4.3.2. Precision.....	13
4.3.3. Recall.....	13
4.3.4. F1-Score.....	14
4.4. Initial Model Performance.....	14
4.5. Optimized Model Performance.....	16
4.6. Computational Efficiency.....	17
4.8. How the Metrics Were Calculated.....	18
4.9. Model Insights.....	19
5. Discussion.....	19
6. Conclusion.....	20
7. References.....	20

1. Abstract

Some machine learning models were developed to accurately identify spam and legitimate emails. The classifier helps users filter unwanted emails, making their inboxes cleaner and safer. Using data from Kaggle [1], the spam detection system effectively blocks harmful content while letting important emails through. This helps users focus on meaningful communications without worrying about spam or fraud.

2. Introduction

Email communication is a vital aspect of modern life, yet it is often disrupted by spam messages that clutter inboxes and pose potential security risks. This project focuses on developing a reliable machine learning-based spam detection system to address this issue. By utilizing a dataset from Kaggle, multiple models were trained and evaluated to identify the most effective approach for distinguishing spam emails from legitimate ones. This report details the methodology, evaluation results, and findings, providing insights into the strengths and limitations of different algorithms. The ultimate goal is to enhance email security and usability through a robust classification system.

3. Methodology

The methodology for this project consists of several key stages, including data preprocessing, feature extraction, model development, and evaluation. Each step is critical for building an accurate and effective spam detection model.

3.1. Data Collection and Preprocessing

The email dataset was sourced from Kaggle, specifically the "Enron Spam Dataset," which includes a variety of messages categorized as spam and ham (legitimate). This dataset serves as the foundation for developing the spam detection model.

3.1.1. Data Cleaning

Data preprocessing is an essential step in preparing the dataset for modeling. The following actions were performed during the cleaning process:

- **Data Extraction and Loading:** The dataset was extracted from the "enron_spam.zip" file using the zipfile module and loaded into a structured format. The load_messages function was used to process the raw text files and append each message along with its associated label (0 for "ham", 1 for "spam") into the dataset.
- **Cleaning the Labels:** The categorical labels "ham" and "spam" were converted to numeric values (0 and 1, respectively) using the map() function for easier model input.
- **Handling Missing Data:** Missing values in the dataset were identified and removed to ensure data quality and prevent model bias due to incomplete data.
- **Message Cleaning:** The clean_message function was applied to preprocess the text data. This function used regular expressions to remove non-alphabetic characters, convert all text to lowercase, and eliminate stop words. These steps ensured that the dataset consisted of clean and uniform text data.
- **Saving Processed Data:** The cleaned data was saved into a new CSV file ("cleaned_enron_dataset.csv") to facilitate easy access during model training.

3.1.2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was conducted to gain insights into the dataset before model training:

- **Message Length Statistics:** Descriptive statistics on the message length were calculated, with the majority of messages being relatively short. A few messages exceeded 31,000 characters, creating a long-tail distribution.

- **Class Distribution:** The dataset was imbalanced, with more "ham" messages than "spam." This was visualized to understand the need for handling class imbalance during model development.
- **Message Length and Class Relationship:** A box plot was used to illustrate the relationship between message length and label, showing that spam messages tended to be shorter.
- **Frequent Words in Cleaned Messages:** The most common words in the cleaned messages were analyzed to understand the frequent themes and topics in the dataset.

3.1.3. Feature Engineering and Selection

Feature engineering plays a vital role in transforming the raw text into meaningful input for the model. The following features were created:

- **Message Length:** The length of each message, measured by the number of characters.
- **Word Count:** The number of words in each cleaned message.
- **Punctuation Count:** The total number of punctuation marks in the original message.
- **Capital Letter Ratio:** The proportion of capital letters to total characters in the original message.
- **Spam Keyword Count:** A count of the number of keywords associated with spam messages, extracted using the Term Frequency-Inverse Document Frequency (TF-IDF) method.

Each feature was selected to capture distinct aspects of spam and non-spam messages, such as verbosity, punctuation usage, and the presence of common spam keywords.

3.2. Model Development

3.2.1. Algorithm Selection

Several machine learning algorithms were selected for model development, including:

- Random Forest
- Logistic Regression
- Naive Bayes
- K-Nearest Neighbors

These algorithms were chosen based on their ability to classify email messages effectively and their widespread use in text classification tasks.

3.2.2. Model Training and Testing

The dataset was split into training and test sets. The model was trained using the training data, and performance was evaluated using the test data. Cross-validation techniques were applied to improve the model's robustness and generalization.

3.3. Model Evaluation and Optimization

3.3.1. Performance Metrics

The performance of each model was evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score

These metrics were crucial in assessing the model's ability to identify spam messages while minimizing false positives and false negatives.

3.3.2. Model Optimization

Hyperparameter tuning was conducted to optimize the performance of the selected models. Techniques like Grid Search and Randomized Search were used to adjust the model parameters for better accuracy and efficiency. The model that demonstrated the best performance across these metrics was selected as the final model.

4. Results

4.1. Data Preprocessing

4.1.1. First Few Rows of the Cleaned Dataset

The dataset has been preprocessed, with each row representing an email or message. The cleaned version of the dataset includes a column for the cleaned messages (Clean_Message) and their corresponding labels (Label), where 0 represents "ham" and 1 represents "spam." Below is a snapshot of the first few rows:

Index	Message	Clean_Message	Label
0	Subject: christmas tree farm pictures\n	subject christmas tree farm pictures	0
1	Subject: vastar resources , inc .\ngary , prod...	subject vastar resources inc gary production h...	0
2	Subject: calpine daily gas nomination\n- calpi...	subject calpine daily gas nomination calpine d...	0
3	Subject: re : issue\nfyi - see note below - al...	subject issue fyi see note already done stella..	0
4	Subject: meter 7268 nov allocation\nfyi .\n- -...	subject meter nov allocation fyi forwarded lau...	0

Table 4.1.1: First few rows of the cleaned dataset

4.1.2. Basic Information About the Cleaned Dataset

The cleaned dataset contains 5,172 messages, all of which are complete (no missing values). The "Clean_Message" column contains text data, while the "Label" column contains numeric values (0 for ham and 1 for spam).

#	Column	Non-Null Count	Dtype
0	Message	5172	object
1	Clean_Message	5172	object
2	Label	5172	int64

Table 4.1.2: Initial Results for Random Forest Classifier

4.1.3. Message Length Statistics

The message lengths in the dataset are mostly short, with a median length of 529 characters. However, there are a few outliers with much longer messages. Below are the descriptive statistics for message lengths.

Name	Message_Length
count	5172.00
mean	1030.15
std	1505.23
min	10.00
25%	238.00
50%	529.00
75%	1214.25
max	31860.00

Table 4.1.3.1: Message length statistics

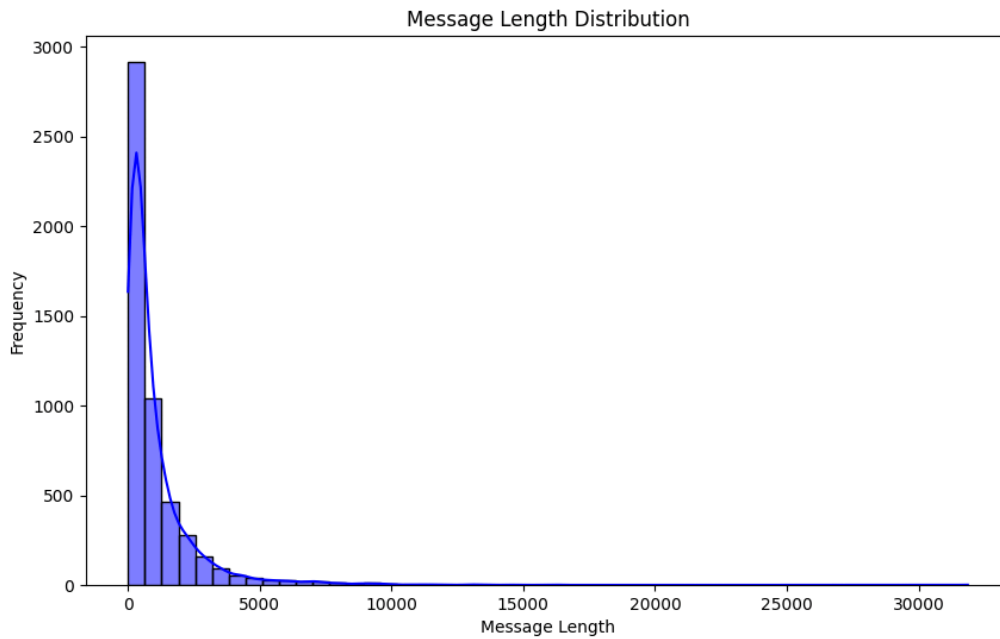


Fig 4.1.3.2: Message Length distribution

4.1.4. Class Distribution

The dataset is imbalanced, with a significantly higher number of "ham" (0) messages compared to "spam" (1) messages. The distribution of classes is shown in the figure below.

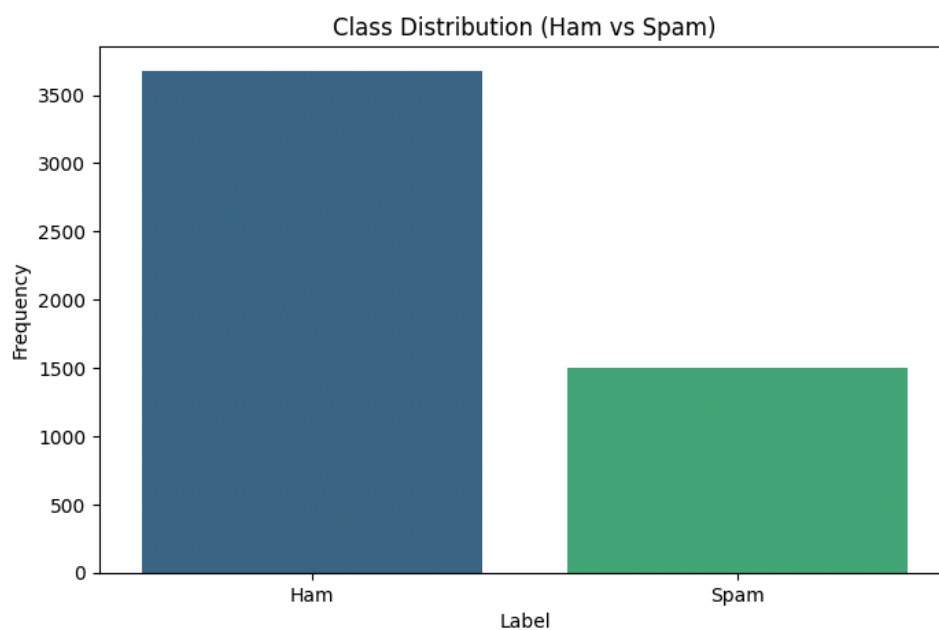


Fig 4.1.4: Class distribution (ham vs spam)

4.1.5. Relationship Between Message Length and Label

A boxplot below illustrates the relationship between message length and labels. Spam messages tend to be shorter, while ham messages have a wider range of lengths.

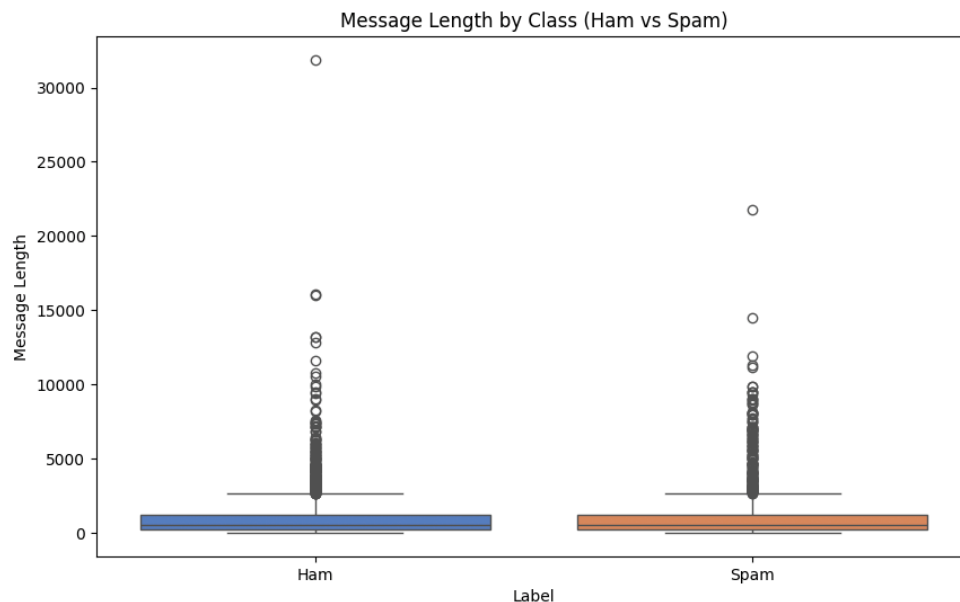


Fig 4.1.5: Message length by class (ham vs spam)

4.1.6. Top 10 Most Frequent Words in Cleaned Messages

The most frequent words in the cleaned messages are shown below. These words provide insights into the common themes in the dataset.

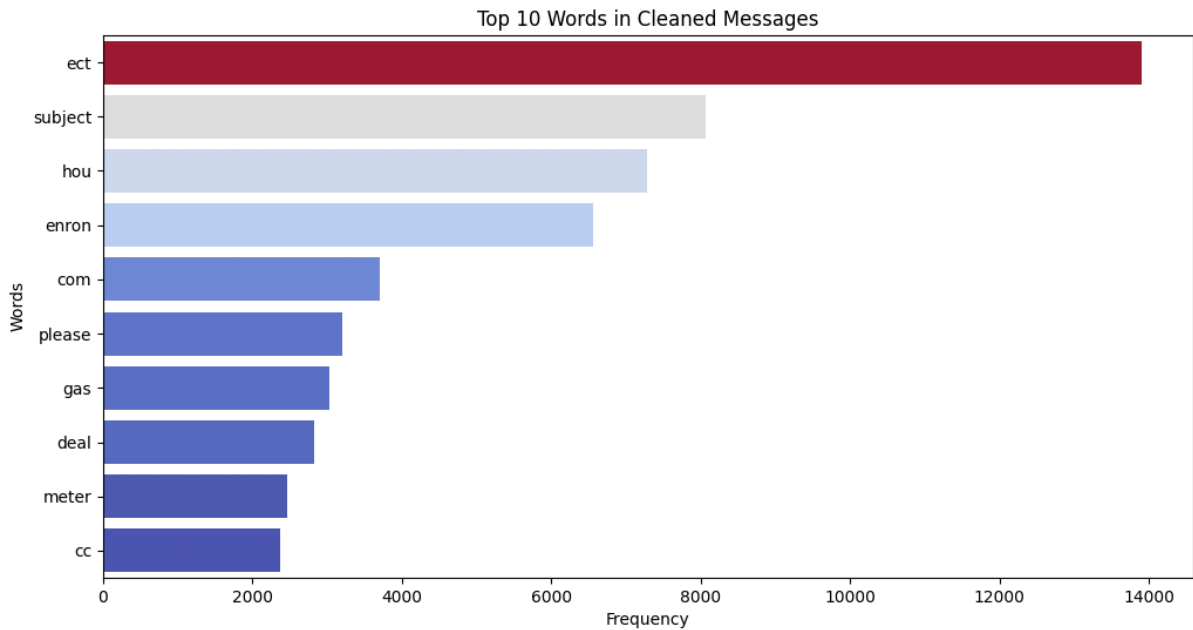


Fig 4.1.6: Top 10 words in cleaned messages

4.2. Feature Engineering

Feature engineering involves transforming raw data into features that better represent the underlying problem, ultimately improving the performance of predictive models. For this project, the following features were engineered:

4.2.1. Message Length

- **Definition:** The number of characters in each message.
- **Rationale:** Spam messages tend to be either unusually short or excessively long.
- **Implementation:** The length of the text in the `Message` column was calculated using the `len()` function.

4.2.2. Word Count

- **Definition:** The number of words in the cleaned message.
- **Rationale:** This feature reflects the level of detail in a message. Spam messages often contain fewer words or redundant repetitions.
- **Implementation:** Text in the `Clean_Message` column was split into words using the `split()` function, and the word count was computed with `len()`.

4.2.3. Punctuation Count

- **Definition:** The total number of punctuation marks in the original message.
- **Rationale:** Spam messages often use excessive punctuation to grab attention or convey urgency (e.g., "Buy Now!!!").
- **Implementation:** Each character in the original message was checked against `string.punctuation` to count punctuation marks.

4.2.4. Capital Letter Ratio

Definition: The proportion of uppercase letters to the total number of characters in the original message.

Rationale: Spam messages may use more uppercase letters to emphasize certain words or phrases.

Implementation: Uppercase letters (`char.isupper()`) were counted and divided by the total message length.

4.2.5. Spam Keyword Count

Definition: The number of spam-specific keywords in each message.

Rationale: Words like "free," "win," or "click" are commonly found in spam messages. Counting these keywords helps distinguish spam from legitimate messages.

Implementation:

- **Spam Keyword Identification:** The most frequently used 50 terms in spam messages were extracted using the TF-IDF (Term Frequency-Inverse Document Frequency) method.
- **Keyword Selection:** The top 10 terms were selected based on importance scores.
- **Counting Process:** The occurrence of these keywords in each message was counted and added as a feature.

4.3. Model Performance Evaluation

The performance of the models was assessed using several key metrics: Accuracy, Precision, Recall, and F1-Score. These metrics provide insights into how well each model performs, particularly in the context of classifying data accurately and efficiently.

4.3.1. Accuracy

- **Definition:** Accuracy measures the proportion of correct predictions (both true positives and true negatives) among the total number of predictions made. It is the ratio of correctly classified samples to the total number of samples.

- **Formula:**

$$Accuracy = \frac{\text{True Predictions}}{\text{Total Number of Samples}}$$

- **Source:** In the provided dataset, each model's accuracy was given. For instance, the Random Forest model achieved an accuracy of **86%**, meaning that 86% of the test samples were correctly classified.

4.3.2. Precision

- **Definition:** Precision measures the proportion of positive predictions (e.g., spam messages) that are actually correct. It answers the question: of all the instances the model classified as positive, how many were truly positive?

- **Formula:**

$$Precision = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Source:** For example, the Random Forest model had a precision of **80%**, meaning that 80% of the messages flagged as spam by the model were indeed spam.

4.3.3. Recall

- **Definition:** Recall measures the model's ability to identify all actual positive instances. In other words, it shows how well the model can detect the true positive cases, such as all spam messages.

- **Formula:**

$$Recall = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Negatives\ (FN)}$$

- **Source:** For instance, the Recall of the Random Forest model is **70%**, meaning the model correctly identified 70% of the actual spam messages.

4.3.4. F1-Score

- **Definition:** The F1-Score is the harmonic mean of precision and recall, providing a balance between the two. It is especially useful when there is a class imbalance in the dataset.

- **Formula:**

$$F1\ Score = 2 \times \left(\frac{Precision \times Recall}{Precision + Recall} \right)$$

- **Source:** For example, the Random Forest model's F1-Score is **75%**, indicating a balanced performance in both precision and recall.

4.4. Initial Model Performance

The metrics presented in this section are based on the initial models before any optimization process:

4.4.1. Random Forest Classifier

	Precision	Recall	F1-Score
0 (ham)	0.88	0.93	0.91
1 (spam)	0.80	0.70	0.75
Accuracy			0.86
Macro Avg	0.84	0.81	0.83
Weighted Avg	0.86	0.86	0.86

Table 4.4.1. : Initial Results for Random Forest Classifier

Random Forest exhibited strong performance overall but struggled with recall for spam messages, missing some true positives.

4.4.2. Logistic Regression

	Precision	Recall	F1-Score
0 (ham)	0.80	0.95	0.87
1 (spam)	0.76	0.43	0.55
Accuracy			0.79
Macro Avg	0.78	0.69	0.71
Weighted Avg	0.79	0.80	0.78

Table 4.4.2. : Initial Results for Logistic Regression

The model's low recall for spam messages indicated limitations in capturing complex relationships due to its linear assumptions.

4.4.3. Naive Bayes

	Precision	Recall	F1-Score
0 (ham)	0.80	0.58	0.68
1 (spam)	0.39	0.65	0.48
Accuracy			0.60
Macro Avg	0.59	0.62	0.58
Weighted Avg	0.68	0.60	0.62

Table 4.4.3. : Initial Results for Naive Bayes

Naive Bayes struggled due to its independence assumptions, resulting in inconsistent performance.

4.4.4. Nearest Neighbors

	Precision	Recall	F1-Score
0 (ham)	0.80	0.90	0.85
1 (spam)	0.66	0.46	0.55
Accuracy			0.78

Macro Avg	0.73	0.68	0.70
Weighted Avg	0.76	0.78	0.76

Table 4.4.4. : Initial Results for k-Nearest Neighbors

KNN's performance was hindered by sensitivity to high-dimensional data and class imbalance.

4.5. Optimized Model Performance

After applying optimization techniques, including hyperparameter tuning and feature engineering, the optimized model's performance was evaluated. The metrics achieved by the optimized model are as follows:

4.5.1. Random Forest Classifier (Grid Search)

	Precision	Recall	F1-Score
0 (ham)	0.87	0.94	0.90
1 (spam)	0.82	0.64	0.72
Accuracy			0.86
Macro Avg	0.85	0.79	0.81
Weighted Avg	0.85	0.86	0.85

Table 4.5.1. : Optimized Results for Random Forest Classifier (Grid Search)

4.5.2. Random Forest Classifier (Randomized Search)

	Precision	Recall	F1-Score
0 (ham)	0.88	0.94	0.91
1 (spam)	0.82	0.69	0.75
Accuracy			0.87
Macro Avg	0.85	0.81	0.83
Weighted Avg	0.86	0.87	0.86

Table 4.5.2. : Optimized Results for Random Forest Classifier (Randomized Search)

4.5.3. Logistic Regression

	Precision	Recall	F1-Score
0 (ham)	0.81	0.95	0.87
1 (spam)	0.78	0.44	0.56
Accuracy			0.80
Macro Avg	0.79	0.69	0.72
Weighted Avg	0.80	0.80	0.78

Table 4.5.3. : Optimized Results for Logistic Regression

4.5.4. Naive Bayes

	Precision	Recall	F1-Score
0 (ham)	0.80	0.58	0.68
1 (spam)	0.39	0.65	0.48
Accuracy			0.78
Macro Avg	0.59	0.62	0.58
Weighted Avg	0.68	0.60	0.62

Table 4.5.4. : Optimized Results for Naive Bayes

4.5.5. Nearest Neighbours

	Precision	Recall	F1-Score
0 (ham)	0.81	0.92	0.86
1 (spam)	0.72	0.47	0.57
Accuracy			0.79
Macro Avg	0.76	0.70	0.72
Weighted Avg	0.78	0.79	0.78

Table 4.5.5. : Optimized Results for k-Nearest Neighbors

4.6. Computational Efficiency

- **Random Forest:** While achieving the best results, it required significant training and optimization time (~257 seconds).
- **Logistic Regression and Naive Bayes:** Both were computationally efficient but showed limited potential for improvement.
- **KNN:** Minimal training time but higher computational cost during prediction due to instance-based methods.

4.7. Error Analysis

- **Random Forest:** Occasional misclassification of ham as spam due to overlapping feature distributions.
- **Logistic Regression:** Low recall caused by class imbalance and linear assumptions.
- **Naive Bayes:** Dependency on the feature independence assumption hindered performance.
- **KNN:** Sensitivity to high-dimensional data reduced overall effectiveness.

4.8. How the Metrics Were Calculated

To calculate these metrics, we need to understand the following concepts:

- **True Positives (TP):** The number of spam messages correctly predicted as spam.
- **False Positives (FP):** The number of non-spam messages incorrectly predicted as spam.
- **True Negatives (TN):** The number of non-spam messages correctly predicted as non-spam.
- **False Negatives (FN):** The number of spam messages incorrectly predicted as non-spam.

Using these values, the metrics are derived to evaluate each model's performance. For example, the Random Forest model showed:

- **Accuracy:** 86%, meaning 86% of the total test set was classified correctly.

- **Precision:** 80%, indicating that 80% of the messages predicted as spam were actually spam.
- **Recall:** 70%, meaning the model detected 70% of the actual spam messages.
- **F1-Score:** 75%, reflecting a balanced combination of precision and recall.

4.9. Model Insights

- **Random Forest** outperforms the other models across all metrics, demonstrating its effectiveness in correctly classifying spam messages, with a precision of 80%, recall of 70%, and F1-Score of 75%.
- **Logistic Regression** achieves decent precision (76%) but has a relatively low recall of 43%, indicating that it misses a large number of spam messages. This issue could stem from a class imbalance, where non-spam messages far outnumber the spam messages, which could make the model more inclined to classify most instances as non-spam.
- **Naive Bayes** has a very low precision (39%), meaning it often misclassifies non-spam messages as spam. While its recall is relatively high (65%), indicating it detects most of the actual spam messages, the precision issue still limits its overall performance.
- **k-Nearest Neighbors (KNN)** performs similarly to Logistic Regression, with moderate precision (66%) and recall (46%), indicating some capability to detect spam but with room for improvement.

These insights suggest that while the Random Forest model is the most reliable, other models such as Logistic Regression and Naive Bayes would benefit from further tuning or alternative approaches to improve their ability to correctly classify spam messages, especially with regard to precision and recall.

5. Discussion

We applied four machine learning models Random Forest, Logistic Regression, Naive Bayes, and k-NN to classify emails as spam or ham. The models were evaluated using key metrics such as accuracy, precision, recall, and F1-score.

The Random Forest model performed the best, achieving an accuracy of 86% and precision of 80%. However, issues like class imbalance, where there are significantly more ham emails than spam, affected the performance of some models, particularly Logistic Regression and Naive Bayes. The lower recall and precision in these models suggest that they struggled to correctly identify spam messages.

Exploratory Data Analysis (EDA) revealed that spam messages tend to be shorter in length compared to ham messages. The class distribution was heavily imbalanced, which may have contributed to the challenges faced by some models. Additionally, the frequent occurrence of certain words in both ham and spam messages indicates that feature engineering could improve model performance further.

Future work could explore advanced techniques such as deep learning models or ensemble methods to enhance the classification accuracy and address the class imbalance issue.

6. Conclusion

We applied four machine learning models Random Forest, Logistic Regression, Naive Bayes, and k-NN to classify email messages as spam or ham. After preprocessing the dataset and performing exploratory analysis, Random Forest achieved the best results, with an accuracy of 86% and a precision of 80%. However, there were challenges, such as class imbalance, which affected the recall and precision of some models, particularly Logistic Regression and Naive Bayes.

This project highlights the importance of addressing class imbalance and feature engineering for improving model performance. Future work could focus on more advanced techniques, such as deep learning or ensemble methods, to enhance the accuracy of spam detection.

7. References

[1] <https://www.kaggle.com/datasets/wanderfj/enron-spam>